

ASSIGNMENT 1

Name: Aditya Kumar

Roll: 2501CS19

Github repo link:

[https://github.com/adityainputs/Optimization Techniques Assignment1_CS2105](https://github.com/adityainputs/Optimization_Techniques_Assignment1_CS2105)

1.) BIG M METHOD

Problem statement:

Minimize $z = -3x_1 + x_2 + x_3$

Subject to:

$$x_1 - 2x_2 + x_3 \leq 11$$

$$-4x_1 + x_2 + 2x_3 \geq 3$$

$$2x_1 - x_3 = -1$$

$$x_1, x_2 \geq 0, i = 1, 2, 3.$$

BigM code snapshots:

```

bigM.py > ...
1  import numpy as np
2
3  def print_tableau(tableau, iteration):
4      print(f"\n--- Iteration {iteration} ---")
5      # Formatting to make the tableau look neat in the output
6      print(np.array_str(tableau, precision=2, suppress_small=True))
7
8  def solve_specific_big_m():
9      # M is a very large positive number used as a penalty
10     M = 1000.0
11
12     # Variables order: [x1, x2, x3, s1, s2, a1, a2, RHS]
13     # Initial Table setup based on standard form:
14     # Row 0:  $x_1 - 2x_2 + x_3 + s_1 = 11$ 
15     # Row 1:  $-4x_1 + x_2 + 2x_3 - s_2 + a_1 = 3$ 
16     # Row 2:  $-2x_1 + 0x_2 + x_3 + a_2 = 1$ 
17
18     tableau = np.array([
19         [ 1.0, -2.0, 1.0, 1.0, 0.0, 0.0, 0.0, 11.0],
20         [-4.0, 1.0, 2.0, 0.0, -1.0, 1.0, 0.0, 3.0],
21         [-2.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0, 1.0],
22         [-3.0, 1.0, 1.0, 0.0, 0.0, M, M, 0.0] # Initial Objective row for Max Z'
23     ])
24
25     # Drive out the artificial variables (a1, a2) from the objective row
26     # Obj Row = Obj Row - M*(Row 1) - M*(Row 2)
27     tableau[3] = tableau[3] - M * tableau[1] - M * tableau[2]
28
29     iteration = 0
30     print_tableau(tableau, iteration)
31
32     while True:
33         # Check if optimal (all values in objective row  $\geq 0$  for this setup)
34         # We look at all columns except the RHS
35         if all(tableau[3, :-1]  $\geq -1e-5$ ):
36             print("\nOptimal solution found!")

```

```

35     if all(tableau[3, :-1]  $\geq -1e-5$ ):
36         print("\nOptimal solution found!")
37         break
38
39     # Entering variable: Most negative value in the objective row
40     enter_col = np.argmin(tableau[3, :-1])
41
42     # Leaving variable: Minimum positive ratio test (RHS / entering column)
43     ratios = []
44     for i in range(3): # We have 3 constraint rows
45         if tableau[i, enter_col]  $> 1e-5$ :
46             ratios.append(tableau[i, -1] / tableau[i, enter_col])
47         else:
48             ratios.append(float('inf'))
49
50     leave_row = np.argmin(ratios)
51     if ratios[leave_row] == float('inf'):
52         print("Unbounded solution.")
53     return
54
55     # Pivot Operations
56     pivot = tableau[leave_row, enter_col]
57     tableau[leave_row] = tableau[leave_row] / pivot
58
59     # Row operations to make other elements in entering column 0
60     for i in range(4): # 3 constraints + 1 objective row
61         if i != leave_row:
62             tableau[i] = tableau[i] - tableau[i, enter_col] * tableau[leave_row]
63
64     iteration += 1
65     print_tableau(tableau, iteration)
66
67     # Since we maximized Z' = -z, the minimum z is -Z' (the RHS of the objective row)
68     min_z = tableau[3, -1]
69     print(f"\nOptimal Objective Value (Minimize z): {round(min_z, 2)}")

```

```

60         for i in range(4): # 3 constraints + 1 objective row
61             if i != leave_row:
62                 tableau[i] = tableau[i] - tableau[i, enter_col] * tableau[leave_row]
63
64             iteration += 1
65             print_tableau(tableau, iteration)
66
67         # Since we maximized Z' = -z, the minimum z is -Z' (the RHS of the objective row)
68         min_z = tableau[3, -1]
69         print(f"\nOptimal Objective Value (Minimize z): {round(min_z, 2)}")
70         print("Check the RHS values in the final tableau to find the values for the basic variables.")
71
72 if __name__ == "__main__":
73     solve_specific_big_m()

```

Output Snapshot:

```

PS D:\Desktop\1\OT> python bigM.py

--- Iteration 0 ---
[[ 1. -2. 1. 1. 0. 0. 0. 11.]
 [ -4. 1. 2. 0. -1. 1. 0. 3.]
 [ -2. 0. 1. 0. 0. 0. 1. 1.]
 [ 5997. -999. -2999. 0. 1000. 0. 0. -4000.]]

--- Iteration 1 ---
[[ 3. -2. 0. 1. 0. 0. -1. 10.]
 [ 0. 1. 0. 0. 0. -1. 1. -2.]
 [ -2. 0. 1. 0. 0. 0. 1. 1.]
 [ -1. -999. 0. 0. 1000. 0. 2999. -1001.]]

--- Iteration 2 ---
[[ 3. 0. 0. 1. -2. 2. -5. 12.]
 [ 0. 1. 0. 0. -1. 1. -2. 1.]
 [ -2. 0. 1. 0. 0. 0. 1. 1.]
 [ -1. 0. 0. 0. 1. 999. 1001. -2.]]

--- Iteration 3 ---
[[ 1. 0. 0. 0.33 -0.67 0.67 -1.67 4. ]
 [ 0. 1. 0. 0. -1. 1. -2. 1. ]
 [ 0. 0. 1. 0.67 -1.33 1.33 -2.33 9. ]
 [ 0. 0. 0. 0.33 0.33 999.67 999.33 2. ]]

Optimal solution found!

Optimal Objective Value (Minimize z): 2.0
Check the RHS values in the final tableau to find the values for the basic variables.
PS D:\Desktop\1\OT>

```

VAM and MODI method:

Problem Statement:

CHAPTER 4: THE TRANSPORTATION PROBLEM (BALANCED)

Q.1. Given the cost matrix, supply, and demand data below for a transportation problem with three origins (O_1, O_2, O_3) and three destinations (D_1, D_2, D_3):

- 1) Determine the initial basic feasible solution using Vogel's Approximation Method (VAM).
- 2) Using the Modified Distribution (MODI) method, test for optimality, and determine the optimal transportation schedule with the minimum total cost.

Origins/ Destinations	D_1	D_2	D_3	Supply
O_1	2	2	3	10
O_2	4	1	2	15
O_3	1	3	1	40
Demand	20	15	30	(Total = 65)

CHECK: Total Supply = $10 + 15 + 40 = 65$,
Total Demand = $20 + 15 + 30 = 65$.
Verification: Total Supply = Total Demand = 65.
The problem is BALANCED.

Code Snapshots:

```

vam_modipy > ...
1  import numpy as np
2
3  def vam(costs, supply, demand):
4      supply_copy = supply.copy().astype(float)
5      demand_copy = demand.copy().astype(float)
6      allocations = np.zeros(costs.shape)
7
8      print("--- Vogel's Approximation Method (VAM) ---")
9      while sum(supply_copy) > 0 and sum(demand_copy) > 0:
10         # Penalties
11         row_penalties = []
12         for i in range(costs.shape[0]):
13             if supply_copy[i] > 0:
14                 valid_costs = [costs[i][j] for j in range(costs.shape[1]) if demand_copy[j] > 0]
15                 if len(valid_costs) > 1:
16                     sorted_costs = sorted(valid_costs)
17                     row_penalties.append((sorted_costs[1] - sorted_costs[0], i))
18                 elif len(valid_costs) == 1:
19                     row_penalties.append((valid_costs[0], i))
20             else:
21                 row_penalties.append((-1, i))
22
23         col_penalties = []
24         for j in range(costs.shape[1]):
25             if demand_copy[j] > 0:
26                 valid_costs = [costs[i][j] for i in range(costs.shape[0]) if supply_copy[i] > 0]
27                 if len(valid_costs) > 1:
28                     sorted_costs = sorted(valid_costs)
29                     col_penalties.append((sorted_costs[1] - sorted_costs[0], j))
30                 elif len(valid_costs) == 1:
31                     col_penalties.append((valid_costs[0], j))
32             else:
33                 col_penalties.append((-1, j))
34

```

```

35         # maximum penalty
36         max_row_pen = max(row_penalties, key=lambda x: x[0])
37         max_col_pen = max(col_penalties, key=lambda x: x[0])
38
39         if max_row_pen[0] >= max_col_pen[0]:
40             row = max_row_pen[1]
41             valid_cols = [(costs[row][j], j) for j in range(costs.shape[1]) if demand_copy[j] > 0]
42             col = min(valid_cols, key=lambda x: x[0])[1]
43         else:
44             col = max_col_pen[1]
45             valid_rows = [(costs[i][col], i) for i in range(costs.shape[0]) if supply_copy[i] > 0]
46             row = min(valid_rows, key=lambda x: x[0])[1]
47
48         # Allocation
49         qty = min(supply_copy[row], demand_copy[col])
50         allocations[row][col] = qty
51         supply_copy[row] -= qty
52         demand_copy[col] -= qty
53
54         print("Initial Basic Feasible Solution (VAM) Allocations:")
55         print(allocations)
56
57         total_cost = np.sum(allocations * costs)
58         print(f"Total Initial Cost (VAM): {total_cost}\n")
59         return allocations
60
61     def modi(costs, allocations):
62         print("--- MODI Method (Optimality Check) ---")
63         rows, cols = costs.shape
64         u = [None] * rows
65         v = [None] * cols
66         u[0] = 0 # Set first u to 0 to begin
67

```

```

68 # Finding basic cells (allocations > 0)
69 basic_cells = [(i, j) for i in range(rows) for j in range(cols) if allocations[i][j] > 0]
70
71 # (Ui + Vj = Cij for basic cells)
72 while None in u or None in v:
73     progress = False
74     for i, j in basic_cells:
75         if u[i] is not None and v[j] is None:
76             v[j] = costs[i][j] - u[i]
77             progress = True
78         elif v[j] is not None and u[i] is None:
79             u[i] = costs[i][j] - v[j]
80             progress = True
81
82     # Degeneracy handler: If we get stuck, assign 0 to the next available 'u'
83     if not progress and (None in u or None in v):
84         for i in range(rows):
85             if u[i] is None:
86                 u[i] = 0
87                 break
88
89     print(f"u values (Sources): {u}")
90     print(f"v values (Destinations): {v}")
91
92     # (Deltaij = Cij - (Ui + Vj))
93     is_optimal = True
94     for i in range(rows):
95         for j in range(cols):
96             if allocations[i][j] == 0:
97                 delta = costs[i][j] - (u[i] + v[j])
98                 if delta < 0:
99                     is_optimal = False
100                     print(f"Negative penalty found at non-basic cell ({i},{j}): {delta}")
101
102     if is_optimal:
103         print("\nAll opportunity costs >= 0. The current solution is OPTIMAL.")
104         print(f"Optimal Transportation Cost: {np.sum(allocations * costs)}")
105     else:
106         print("\nSolution is NOT optimal yet. A closed loop must be formed to shift allocations.")
107
108 if __name__ == "__main__":
109     # problem matrix
110     costs = np.array([
111         [2, 2, 3],
112         [4, 1, 2],
113         [1, 3, 1]
114     ])
115
116     supply = np.array([10, 15, 40])
117     demand = np.array([20, 15, 30])
118
119     print("Problem Matrix:")
120     print("Costs:\n", costs)
121     print("Supply:", supply)
122     print("Demand:", demand, "\n")
123
124     # VAM followed by MODI
125     initial_allocations = vam(costs, supply, demand)
126     modi(costs, initial_allocations)

```

Output snapshot:

```
Problems | Terminal | Output | Debug Console | Ports
PS D:\Desktop\1\OT> python vam_modi.py
Problem Matrix:
Costs:
[[2 2 3]
 [4 1 2]
 [1 3 1]]
Supply: [10 15 40]
Demand: [20 15 30]

--- Vogel's Approximation Method (VAM) ---
Initial Basic Feasible Solution (VAM) Allocations:
[[10.  0.  0.]
 [ 0. 15.  0.]
 [10.  0. 30.]]
Total Initial Cost (VAM): 75.0

--- MODI Method (Optimality Check) ---
u values (Sources): [0, 0, np.int64(-1)]
v values (Destinations): [np.int64(2), np.int64(1), np.int64(2)]

All opportunity costs >= 0. The current solution is OPTIMAL.
Optimal Transportation Cost: 75.0
PS D:\Desktop\1\OT>
```